CSE1242 – Selective and Repetitive Structures in C

Dr. Sanem Arslan Yılmaz

Some of the slides are from:
CMPE150 – Boğaziçi University
Deitel & Associates

Agenda

- Algorithms & Pseudocode
- Flowcharts
- If/Else statements
- Ternary operator
- Switch statement
- Types of loops
  - for loop
  - while loop
  - do-while loop
- Continue/Break statements

Introduction

- Before writing a program to solve a particular problem, we must have a thorough understanding of the problem and a carefully planned solution approach.

Algorithms

- The solution to any computing problem involves executing a series of actions in a specific order.
- A procedure for solving a problem in terms of
  - the actions to be executed, and
  - the order in which these actions are to be executed

  is called an algorithm.
- Correctly specifying the order in which the actions are to be executed is important.

Algorithms (Cont.)

- Consider the "rise-and-shine algorithm" followed by one junior executive for getting out of bed and going to work:
  - (1) Get out of bed,
  - (2) take off pajamas,
  - (3) take a shower,
  - (4) get dressed,
  - (5) eat breakfast,
  - (6) carpool to work.
- This routine gets the executive to work well prepared to make critical decisions.
- Specifying the order in which statements are to be executed in a computer program is called program control.

Pseudocode

- Pseudocode is an artificial and informal language that helps you develop algorithms.
- Pseudocode is similar to everyday English; it's convenient and user friendly although it's not an actual computer programming language.
- Pseudocode programs are *not* executed on computers.
- Rather, they merely help you "think out" a program before attempting to write it in a programming language like C.

Pseudocode (Cont.)

- A carefully prepared pseudocode program may be converted easily to a corresponding C program.
- Pseudocode consists only of action and decision statements—those that are executed when the program has been converted from pseudocode to C and is run in C.
- Definitions are not executable statements.
- They’re simply messages to the compiler.

Flowcharts

- A flowchart is a graphical representation of an algorithm or of a portion of an algorithm.
- Like pseudocode, flowcharts are useful for developing and representing algorithms, although pseudocode is preferred by most programmers.
- Flowcharts are drawn using certain special-purpose symbols such as rectangles, diamonds, rounded rectangles, and small circles; these symbols are connected by arrows called flowlines.

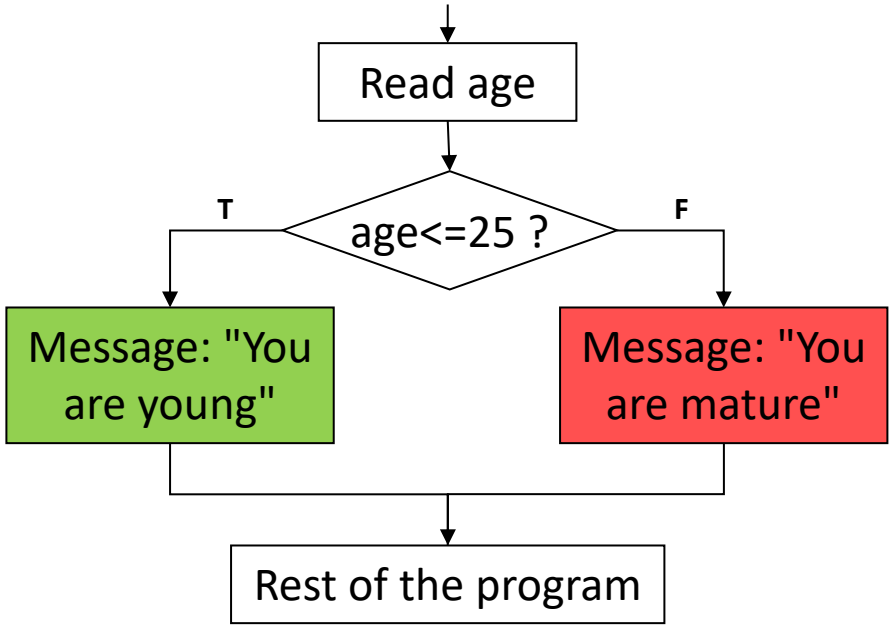
Flowcharts

- We use the rectangle symbol, also called the action symbol, to indicate any type of action including a calculation or an input/output operation.
- Perhaps the most important flowcharting symbol is the diamond symbol, also called the decision symbol, which indicates that a decision is to be made.
- The flowlines indicate the order in which the actions are performed.

If statement

- The "if statement" is used to break the sequential flow of execution.

- Enforces branching in execution according to the result of an expression.
  - There are two possible paths to take.
  - The expression determines which path should be taken.



If statement syntax

- Syntax:

```
if (int_expr)
    stat_block1
else
    stat_block2
```

where **stat_block** is one of the following:

- a single statement `stat;`
- the null statement `;`
- a group of statements enclosed in braces

```
{
    stat1;
    ...
    statn;
}
```

Do not omit braces in multiline blocks

- Suppose we want to count the students that passed.

```
if (grade > 50){
        printf("Passed.\n");
        number_passed += 1;
}
```

- If we omit the braces this becomes

```
if (grade > 50)
        printf("Passed.\n");
        number_passed += 1;
```

- Equivalent to

```
if (grade > 50){
        printf("Passed.\n");
}
number_passed ++;
```

- The number is increased for any grade.

No semicolon after if or else

- A common mistake is to put a semicolon at the end of the ***if*** line:

```
if (grade > 50);
        printf("Passed.\n");
```

- This is valid code; no syntax error. It is the same as:

```
if (grade > 50)
{
        /* empty block */
}
printf("Passed.\n");
```

- Thus, "Passed" is printed for any grade. Not what we want!

If statement example

- Read one character as input; check if it is a digit; if so, convert it to an integer and display the number that is two times the input number; o/w display an error message.

```
char ch;   int num;

scanf("%c", &ch);
if (('0' <= ch) && (ch <= '9')){
   num = ch - '0';
   printf("Two times the input is %d \n", 2 * num);
}
else
   printf("Input is not a digit! \n");
```

Ternary operator

- Ternary operator is similar to the "if" statement. But it is an operator, not a statement.

- Syntax:

```
<condition> ? <expr_1> : <expr_2>
```

- Eg:

```
a = (b > c) ? b : c;
k = (n != 0) ? m/n : 0;
```

Switch statement

- If you have multiple cases depending on different values of the same integer expression, switch is easier to use.

- Syntax:

```
switch (int_expr)
{  case constant_int_value1: stat(s);
   case constant_int_value2: stat(s);
   ...
   default: stat(s);
}
```

- You may have zero or more statements in each case.

Break statement

- Switch statement actually gathers many statements of several cases.
  - The case labels denote the specific statement from which the execution of this group of statements begins.
  - All statements till the end of the group are executed sequentially.
- To separate the cases, **`break`** statement is used.
  - **`break`** breaks the sequential execution of the statements and immediately jumps to the end of the switch statement.

Switch statement example

- Define the days of the week as an enumerated type and display them as strings.

```
enum day_type {MON=1,TUE,WED,THU,FRI,SAT,SUN} day;

scanf("%d", &day);
switch (day)
{  case SUN: printf("Sunday\n"); break;
   case WED: printf("Wednesday\n"); break;
   case TUE: printf("Tuesday\n"); break;
   case THU: printf("Thursday\n"); break;
   case FRI: printf("Friday\n"); break;
   case SAT: printf("Saturday\n"); break;
   case MON: printf("Monday\n"); break;
   default:  printf("Incorrect day\n"); break;
}
```

Repetitive Structures in C

Types of loops

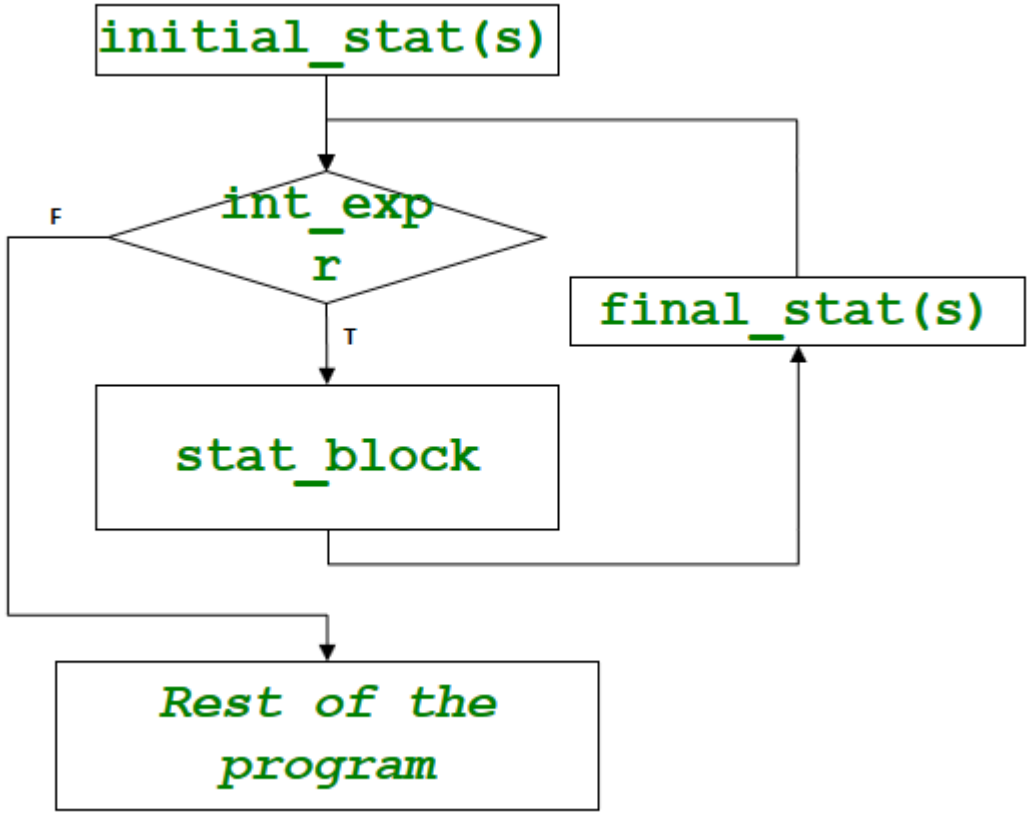
- There are three types of loops:
  - "for" loop
  - "while" loop
  - "do-while" loop

- You can implement anyone of these loops using the others (and possibly an additional if statement and duplication of some statements).
  - The idea is to use the type of loop that is best suited for your problem.

For loop syntax

- Syntax:

```
for (initial_stat(s);int_expr; final_stat(s))
    stat_block
```



For loop

- Note that initial and final statements as well as the integer expression are optional according to the syntax.
  - If **`initial_stat(s)`** is missing, start directly with the comparison.
  - If **`final_stat(s)`** is missing, go directly to the comparison after the execution of the statement block.
  - If **`int_expr`** is missing, the comparison is always true.
    - Make use of the **`break`** statement.

For loop – Example I

- Find a^b. (**a** and **b** integers)

```
int a, b, result=1, i;

scanf("%d %d", &a, &b);

for (i=0; i<b; i++)
        result *= a;

printf("Result:%d",result);
```

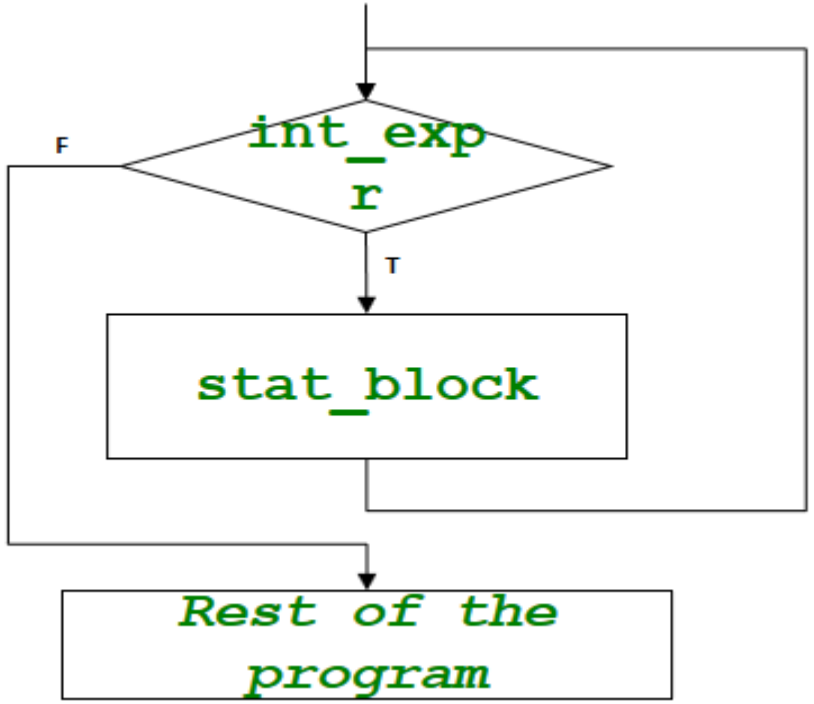
Output
3
5
Result:243

While loop syntax

- Use while loop if the statement block should be executed as long as a condition holds.

- Syntax:

```
while (int_expr)
    stat_block
```



While loop Example

- Find the average of a sequence of integers terminated with a negative value.

```
int sum=0, n, count=0;
float avg;

scanf("%d", &n);
while (n>=0)
{
        sum += n;
        count++;
        scanf("%d", &n);
}
avg = (count) ? (float)sum/count : 0;
printf("Average:%f",avg);
```

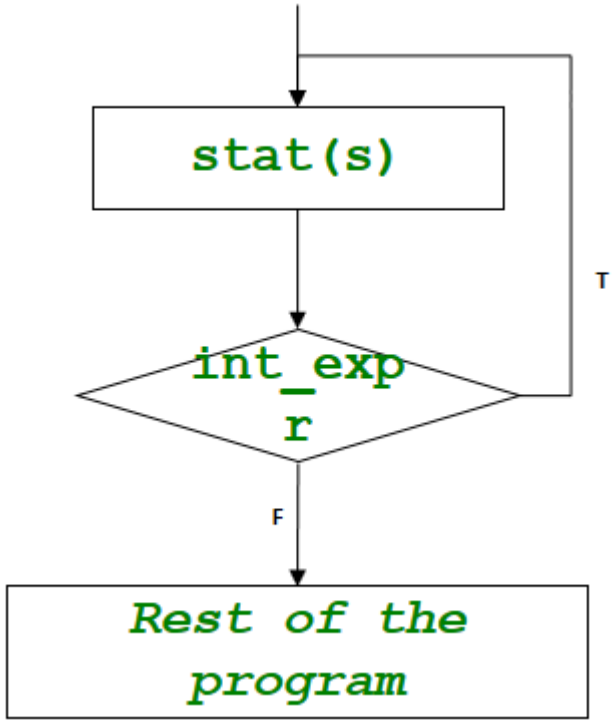
Output

```
1
2
3
4
5
Average:3.000000
```

Do-while loop syntax

- Similar to while loop.
  - Only difference: Condition is checked after the execution of every iteration.
- Syntax:

```
do
{
    stat(s)
}
while (int_expr);
```



Break statement

- It is possible to terminate a loop prematurely.

- Remember the **`break`** statement we discussed before.

  - **`break`** breaks the innermost loop or switch statement.

Break - Example

- Break when number is equal to 5

```
int main(void){
   unsigned int x; // declared here so it can be used after loop
   // loop 10 times
   for (x = 1; x <= 10; ++x) {
      // if x is 5, terminate loop
      if (x == 5) {
         break; // break loop only if x is 5
      }
      printf("%u\n", x);
   }
   printf("\nBroke out of loop at x == %u\n", x);
}
```

Output:

```
1
2
3
4

Broke out of loop at x == 5
```

Continue statement

- It is possible to skip the rest of an iteration and continue with the next iteration (if any).
  - In the for loop, `continue` jumps to the final statement.
  - In the while and do-while loops, `continue` jumps to the condition expression.

Continue - Example

- Continue when number is equal to 5

```
int main(void){
    unsigned int x;
   // loop 10 times
   for (x = 1; x <= 10; ++x) {
      // if x is 5, continue with next iteration of loop
      if (x == 5) {
         continue; // skip remaining code in loop body
      }
      printf("%u\n", x); // display value of x
   }
   printf("\nUsed continue to skip printing the value 5");
}
```

Output:

```
1
2
3
4
6
7
8
9
10

Used continue to skip printing the value 5
```

Consider the differences between the following for loops

`for (i=0; i<5; i++)`
`  printf("%d;", i);`
`printf("{%d}",i);`
	`for (i=0; i<=5; i++)`
	`  printf("%d;", i);`
	`printf("{%d}",i);`
		`for (i=1; i<5; i++)`
		`  printf("%d;", i);`
		`printf("{%d}",i);`

0;1;2;3;4;{5}	0;1;2;3;4;5;{6}	1;2;3;4;{5}

`for (i=1; i<=5; ++i)`
`  printf("%d;", i);`
`printf("{%d}",i);`
	`for (■; i<5; i++)`
	`  printf("%d;", i);`
	`printf("{%d}",i);`
		`for (i=0;■; i++)`
		`  printf("%d;", i);`
		`printf("{%d}",i);`

1;2;3;4;5;{6}	Starts from anything ...;3;4;{5} OR MAYBE SOMETHING LIKE {795}	0;1;2;...∞

`for (i=0; i<5;■)`
`  printf("%d;", i);`
`printf("{%d}",i);`
	`for (i=0; i<5;■)`
	`  printf("%d;",i++);`
	`printf("{%d}",i);`
		`for (i=0; i++<5;■)`
		`  printf("%d;", i);`
		`printf("{%d}",i);`

0;0;0;0;...∞	0;1;2;3;4;{5}	1;2;3;4;5;{6}

Examples (cont.)

`for (i=7; i<5; i++)`
`  printf("%d;", i);`
`printf("{%d}",i);`
	`for (i=7;++i<5;■)`
	`  printf("%d;", i);`
	`printf("{%d}",i);`
		`for (i=7;i++<5;■)`
		`  printf("%d;", i);`
		`printf("{%d}",i);`

{7}	{8}	{8}

`for (i=7; ++i<5; ++i)`
`  printf("%d;",++i);`
`printf("{%d}",i);`
	`for (i=0; i<5; ++i)`
	`  printf("%d;",++i);`
	`printf("{%d}",i);`

{8}	1;3;5;{6}

Example: Is it a prime?

- We start by assuming the number is prime.
- We scan every integer smaller than the square root of **n**.
- If any **i** divides **n**, we decide that **n** is not a prime.

```
#include <stdio.h>
int main() {
        int n, i, isprime = 1;
        printf("Enter value: ");
        scanf("%d", &n);

        for(i=2; i*i <= n; i++)
                if (n%i == 0){
                        isprime = 0;
                        break;
                }

        if (isprime)
                printf("%d is a prime\n", n);
        else
                printf("%d is not a prime\n", n);
        return 0;
}
```

Example: Is it a prime?

- Slight modifications:
  - Use an upper loop to read input many times.
  - Use ternary if-else in printf

```
int n, i, isprime = 1;

while( 1 ) {
        printf("Enter value (-1 to exit): ");
        scanf("%d", &n);

        if( n == -1)
                break;

        for(i=2; i*i <= n; i++)
                if (n%i == 0){
                        isprime = 0;
                        break;
                }

        printf("%d is %s a prime\n",
                        n,
                        isprime ? "" : "not");
}
```

Floats and round-off errors

- Suppose we want to print out a table of squares of floatingpoint numbers 1.0, 1.1, … , 2.0. Display two digits after the decimal point.

```
float x, dx = 0.1;
printf("x\tx^2\n");
for(x = 1.0; x<= 2.0; x += dx)
        printf("%.2f\t%.2f\n", x, x*x);
```

Output

x	x^2
1.00	1.00
1.10	1.21
1.20	1.44
1.30	1.69
1.40	1.96
1.50	2.25
1.60	2.56
1.70	2.89
1.80	3.24
1.90	3.61

- The output stops at 1.90 even though we included 2.0 in the range.

Floats and round-off errors

- Let's display 10 digits after the decimal point to see why.

```
float x, dx = 0.1;
printf("x\tx^2\n");
for(x = 1.0; x<= 2.0; x += dx)
   printf("%.10f\t%.10f\n", x, x*x);
```

- The results are off because the floating-point representation is inexact.

- Repeated additions increase the error.

Output

x	x^2
1.0000000000	1.0000000000
1.1000000238	1.2100000381
1.2000000477	1.4400000572
1.3000000715	1.6900001764
1.4000000954	1.9600002766
1.5000001192	2.2500004768
1.6000001431	2.5600004196
1.7000001669	2.8900005817
1.8000001907	3.2400007248
1.9000002146	3.6100008488

Floats and round-off errors

- To be able to display the final value, increase the range by dx

```
float x, dx = 0.1;
printf("x\tx^2\n");
for(x = 1.0; x<= 2.0 + dx; x += dx)
        printf("%f\t%f\n", x, x*x);
```

Output

```
x          x^2
1.000000        1.000000
1.100000        1.210000
1.200000        1.440000
1.300000        1.690000
1.400000        1.960000
1.500000        2.250000
1.600000        2.560000
1.700000        2.890001
1.800000        3.240001
1.900000        3.610001
2.000000        4.000001
```